SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA14 **COURSE NAME: COMPILER DESIGN**

COURSE OUTCOME COVERED

CO2: *Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)*

Assessment Tool Weightage: 20%

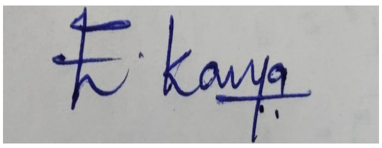
Duration :60 Mins
On Class
QUESTIONS: 25 **Total Marks:50**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I _**E.Kavya(192424365)___**____ *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the student:

Annexure A

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:
id + id
Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:
$S \rightarrow aS \mid b$
Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:
$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:
$S \rightarrow aA \mid b$
$A \rightarrow c \mid \varepsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.
$S \rightarrow Sa \mid b$
Justify your answer.

Q6. Apply left factoring to the grammar:
S → if E then S
S → if E then S else S

Q7. Construct a parse tree for the string:
a + b
using the grammar:
$E \rightarrow E + E$
$E \rightarrow id$

Q8. Write the recursive descent procedures for the grammar:
$S \rightarrow aA$
$A \rightarrow b$

Q9. Given the grammar:
$S \rightarrow AB$
$A \rightarrow a$
$B \rightarrow b$
Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:
id + id

Q11. Identify the handle in the string:
id + id
for the grammar:
$E \rightarrow E + E$
$E \rightarrow id$

Q12. Construct the operator precedence relations for the operators:
+
*
where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:
id * id

Q14. What is an LR parser? Illustrate the parsing process using the grammar:
S → aA
A → b

Q15. Given the grammar:
S → aA
A → b
Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:
S → AB
A → a
B → b

Q17. Construct the augmented grammar for:
S → a

Q18. Find the closure of the LR(0) item:
S' → •S
for the grammar:
S → a

Q19. Check whether the grammar is ambiguous:
E → E + E
E → id
using the string:
id + id + id

Q20. Construct a leftmost derivation for the string:
id + id
using the grammar:
E → E + E
E → id

Q21. Apply left factoring to the grammar:
S → aA
S → aB
A → b
B → c

Q22. Find FIRST(A) for the grammar:
A → x
A → y

Q23. Determine whether the grammar is suitable for predictive parsing.
S → aA | aB
A → b

B → c
Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:
S → aS
S → b

Q25. Build the predictive parsing table for the grammar:
S → aS
S → b
Show the table entries for the terminals:
a, b, $

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Short Answer Test CO2AT2

CSA1408 - Compiler design

6/8/26

Name: E. Kavya

Reg No: 192424365

1) Lexical Analyzer

Given id+id

'+' is valid expression

'id' is valid expression

'id' is valid expression

So, id + id is valid expression

2) $S \rightarrow aS|b$ string belongs to aaab

$S \rightarrow aS$

$S \rightarrow aSb$

$S \rightarrow aaaS$

$S \rightarrow aaaSb$ so, it belongs to language using leftmost derivation

3) $A \rightarrow bA'$

$A' \rightarrow aA'/\varepsilon$

4) First (S) = {a, b}

first (A) = {c, ε}

5) Given $S \rightarrow Sa|b$

$S \rightarrow Sa$, $S \rightarrow ab$ both get a

So, it is not suitable for top- down parsing

Given

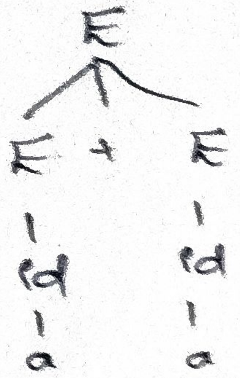
6) S → if E then S

S → if E then S else S

E → ~~if~~E then ~~S/x~~ SS'

x → else S/ε

7) E → E + E



8) S → aA

A → b

```
void();
{
if( ~~ahead~~ input ==a);
  input++;
}
();
```

9)

Non- telmenal	a
S	AB
AB	a
B	b

10) Given

id+id

Step	Stack	impact	Action
1.	id	+id	Shift
2.	E	+id	Reduce id→E
3.	E+	id	Sheft

11) id + id

E → E + E

id + id

E → id

= id

12.) + < *

* > +

13) Given id * id

$	id*id	shift id
id	* id	id → E
E	* id	shift *
E*	id	shift id
E*id	ε	id → E
E*E	ε	Accept

14)

LR Parser is bottom-up parser input is from left to right

S → aA
A → b
= ab$

16) Follow(A) = {AB}

follow(A) = {a}

follow(B) = {b}

So, Follow (A) = Follow (B) = {~~a~~b}

17) Given
S → a

S' → a

18) Given

S' → .S

for S → a

S' → .a

19) id + id + id

(id + id) + id , id + (id + id

It gets two parse trees so, it is ambiguous

20) id + id

E → E + E

E → id + E

E → id + id

E → E + E

E → ~~E~~ id + E

→ id + id

21) Geven

S → aA

S → aB

A → b

B → c

S → A|x

x →

22) A = {x, y}

23) No et es not suitable for predectеve parseng because et gets S → aA ; S → aB both gets a.

24)

```
Procedure c) ;
 {
  et ( enput = = a) ;
      a( ) ;
 }
 {else
 et ( enput = = b) ;
 }
```

25)

Non-termenal	a
S	as
S	b

Stack	Input Buffer	Acteon
$	aA$	shett a
$a	A$	shett A
$aA	$	x

15)

S' → S,

construct LR(0) etems

fend the closure & Goto sets

Bueld the LR(0) automateon

compute follow sets

Full Acteon and Goto tables

Mark, shett & reduce ~~the acto~~ and accept entrees.